

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №3
«Контейнеризация написанного приложения средствами
Docker»

Выполнила:

Олейник Полина

Группа 2.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Реализовать контейнеризацию приложения: Dockerfile для каждого сервиса, общий docker-compose.yml, сетевое взаимодействие между сервисами.

Тема проекта: вариант 1 – приложение для бронирования столиков в ресторанах. Работа выполнена поверх результата ЛР2 и Д35 (три микросервиса, gateway, RabbitMQ).

Ход работы

1. Что изменилось по сравнению с ЛР2

В ЛР2 в Docker работали только db и rabbitmq, а auth, restaurants, bookings и gateway были обычными процессами node, которые я запускала вручную через `npm run dev` и которые обращались друг к другу через localhost. В этой работе все шесть частей стека переведены в контейнеры: каждый сервис получил свой Dockerfile, а взаимодействие между ними идёт не через localhost, а через отдельную docker-сеть по именам контейнеров.

2. Dockerfile для каждого сервиса

Для auth, restaurants, bookings и gateway написала одинаковый по структуре Dockerfile со сборкой в два этапа (multi-stage build):

```
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY tsconfig.json ./
COPY src ./src
RUN npm run build          # компилирует TypeScript в dist/

FROM node:20-alpine
WORKDIR /app
ENV NODE_ENV=production
COPY package*.json ./
RUN npm ci --omit=dev      # только прод-зависимости
COPY --from=builder /app/dist ./dist
EXPOSE 3001
CMD ["node", "dist/app.js"]
```

На первом этапе ставятся все зависимости и собирается TypeScript в обычный JavaScript – команда `npm run build` уже была в `package.json` с ЛР2, просто до этого я всегда запускала `dev`, который выполняет `.ts`-файлы на лету через `tsx`, без отдельной сборки. Второй этап – чистый образ, в который копируется только результат сборки (`dist/`) и ставятся

зависимости без `typescript/tsx`: образ получается меньше и ближе к тому, что реально было бы в продакшене. Сам `package.json` при этом не меняла — команды `build` и `start` там уже были готовы.

Рядом с каждым `Dockerfile` добавила `.dockerignore` (по аналогии с `.gitignore`): он не даёт затащить в контекст сборки `node_modules`, `dist`, `.env` и логи — иначе туда попала бы моя локальная установка зависимостей под Windows, а образ собирается для Linux (`node:20-alpine`).

3. `docker-compose.yml`: собственные образы вместо готовых

Для `db` и `rabbitmq` в `docker-compose.yml`, как и в ЛР2, указан готовый образ (`image: postgres:17`, `image: rabbitmq:3-management`). Для четырёх своих сервисов вместо `image:` указала `build:` — путь к папке с соответствующим `Dockerfile`, например:

```
auth:
  build: ./services/auth
```

Раньше этих четырёх сервисов в `docker-compose.yml` не было вообще — они запускались только вручную.

4. Сеть между контейнерами

У каждого контейнера свой собственный `localhost` — если `auth` внутри себя обратится к `localhost:3002`, он попадёт сам в себя, а не в `restaurants`. Чтобы сервисы находили друг друга, добавила в `docker-compose.yml` собственную сеть и подключила к ней все шесть сервисов:

```
networks:
  app-network:
    driver: bridge
```

Внутри этой сети Docker Compose сам регистрирует каждый контейнер под именем его сервиса, поэтому `http://auth:3001` изнутри сети действительно ведёт к контейнеру `auth` — по тому же принципу, по которому доменное имя в интернете превращается в адрес.

Соответственно переопределила адреса зависимостей на внутренние имена сервисов, а не `localhost`:

```
restaurants:
  environment:
    DB_HOST: db
    DB_PORT: 5432
    AUTH_SERVICE_URL: http://auth:3001
    RABBITMQ_HOST: rabbitmq
    RABBITMQ_PORT: 5672
```

не 15433 — это порт только для хоста
не localhost:3001
не 5673

Отдельно пришлось переопределить `APP_HOST=0.0.0.0`. По умолчанию в `.env` каждого сервиса (он нужен для запуска через `npm run dev` без Docker) стоит `APP_HOST=localhost`, а это значит «слушать только себя», даже находясь в общей сети – без этой строки сервис бы поднялся, но не отвечал бы на запросы от соседних контейнеров.

Файлы `.env` при этом не трогала – они остались как в ЛР2, для локального запуска. Значения в блоке `environment:` файла `docker-compose.yml` приоритетнее значений из `env_file:`, поэтому при запуске через Compose побеждают именно они, а при обычном `npm run dev` на хосте по-прежнему используется `.env`.

5. Порядок старта

В ЛР2 порядок запуска я отслеживала вручную – сначала база, потом сервисы, потом `seed.js`, и один раз из-за этого получила ошибку, когда запустила `seed.js` на пару секунд раньше, чем `auth` был готов. Чтобы не повторять это с шестью контейнерами, добавила проверку готовности:

```
db:
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER}"]
    interval: 5s
    retries: 10

auth:
  depends_on:
    db:
      condition: service_healthy
```

`healthcheck` – команда, которую Docker периодически выполняет внутри контейнера, чтобы понять не «процесс жив», а «сервис реально готов принимать запросы» (`pg_isready` для Postgres, аналогичная проверка для RabbitMQ). `depends_on: condition: service_healthy` заставляет `auth` дожидаться именно такого состояния базы, а не просто факта, что контейнер `db` запущен. По той же схеме `restaurants` и `bookings` ждут готовности `db` и `rabbitmq`, а `gateway` стартует последним, когда все три сервиса уже подняты.

6. Какие порты открыты наружу

В ЛР2 все сервисы были доступны с компьютера напрямую, потому что и были обычными хостовыми процессами. В этой работе в

docker-compose.yml проброс порта на хост (ports:) оставлен только у gateway, db и rabbitmq:

```
gateway:
  ports:
    - 8080:8080

auth:
  # ports: не указан — недоступен с хоста, только внутри app-network
```

auth, restaurants и bookings теперь принимают запросы только от других контейнеров этой же сети. Это соответствует правилу, которое уже было в проекте с ЛР2: снаружи должен быть виден только gateway, а не отдельные сервисы напрямую.

7. Побочный эффект: запуск тестовых данных

Из-за закрытых портов перестал работать прежний способ заполнения базы тестовыми данными. scripts/seed.js обращается не только к публичному /auth/register (он прошёл бы через gateway), но и к внутреннему /internal/users/{id}/role, а /internal/* через gateway всегда отвечает 404 по правилу проекта. Раньше это работало в обход, потому что seed.js обращался напрямую к localhost:3001, а порт auth был открыт на хост. Сейчас этого порта снаружи нет.

Решила это запуском seed.js как отдельного одноразового контейнера в той же сети app-network:

```
docker compose run --rm -v "${PWD}/scripts:/app/scripts" \
-e AUTH_SERVICE_URL=http://auth:3001 auth node scripts/seed.js
```

docker compose run поднимает временный контейнер на основе уже собранного образа auth (в нём уже есть Node и всё нужное), --rm удаляет его сразу после выполнения, а -v подключает папку со скриптом внутрь — в самом образе её нет, Dockerfile копировал только src. Такой контейнер тоже подключён к app-network, поэтому достаёт до настоящего auth по имени. Сам файл scripts/seed.js не меняла.

8. Итог изменений

- добавила Dockerfile и .dockerignore в services/auth, services/restaurants, services/bookings, gateway;
- в docker-compose.yml заменила отсутствие записей об этих сервисах на build: ./путь, добавила environment: с адресами

по именам контейнеров и `APP_HOST=0.0.0.0`, `depends_on` с условиями `service_healthy`;

- добавила `healthcheck` в `db` и `rabbitmq`;
- добавила общую сеть `app-network` и подключила к ней все шесть сервисов;
- убрала проброс портов у `auth`, `restaurants`, `bookings`, оставив его только у `gateway`, `db`, `rabbitmq`;
- поменяла способ запуска `scripts/seed.js` – через `docker compose run`, а не напрямую с хоста.

Код самих сервисов – контроллеры, модели, логика очереди из ДЗ5 – не менялся; изменилось только то, как и где эти же процессы запускаются.

Вывод

В ходе работы приложение из пяти отдельно запускаемых частей (три сервиса и `gateway` на хосте, база и очередь в Docker) превращено в единый стек из шести контейнеров, поднимаемых одной командой `docker compose up -d`. Для этого написаны четыре multi-stage Dockerfile, сервисы подключены к общей сети и находят друг друга по имени контейнера вместо `localhost`, настроен порядок старта через `healthcheck` и `depends_on`, а наружу оставлен открытым только `gateway`, как того требует архитектура, принятая ещё в ЛР2. Отдельно решена сопутствующая проблема с запуском тестовых данных, возникшая из-за закрытия портов внутренних сервисов. Дальше этот же стек будет развёрнут на удалённом сервере в ЛР4.